



STL

(Standard template library)

What is STL?

The **Standard Template Library (STL)** is a collection of **generic classes and functions** in C++ that provide **ready-made data structures and algorithms**.

- ❖ Part of C++ Standard Library
- ❖ Uses **templates**
- ❖ Saves time and reduces errors
- ❖ Supports **Object Oriented Programming**
- ❖ STL allows programmers to work with **containers** and **iterators** efficiently.



STL components

- ❖ **Containers**
- ❖ **Iterators**
- ❖ **Algorithms**

Containers in STL

- ❑ In **STL**, Containers are template classes used to store and organize collections of data.
- ❖ Store multiple elements
- ❖ Automatically manage memory
- ❖ Each container stores data in a specific way

Examples

- Arrays
- Vectors
- Lists
- Stack
- Queue

Iterators in STL

❑ Iterators are objects used to access and traverse elements of a container.

- ❖ Work **like pointers**
- ❖ Used to traverse container elements
- ❖ Provide a common way to access data in different containers

Common Iterator Functions

- begin()
- end()
- --
- ++
- * (for pointers)

Algorithms in STL

- ❑ STL Algorithms are predefined functions in C++ that operate on containers.
- ❖ Perform various operations on data (e.g., searching, sorting, merging)
- ❖ Make code efficient and easier to write
- ❖ STL algorithms are part of `<algorithm>` header.

Common Algos

- `sort()`
- `count()`
- `reverse()`

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main()
{
    vector<int> v = { 4, 2, 5, 3, 1 };
    vector<int>::iterator it; // declare iterator

    // Original vector( dynamic array )
    cout << "Original vector: ";
    for (it = v.begin(); it != v.end(); it++)
    {
        cout << *it << " ";
    }
    cout << endl;

    // sort()
    sort(v.begin(), v.end());
    cout << "After sort(): ";
    for (it = v.begin(); it != v.end(); it++)
    {
        cout << *it << " ";
    }
    cout << endl;
```

```
// count()
int c = count(v.begin(), v.end(), 2);
cout << "Count of 2: " << c << endl;

// reverse()
reverse(v.begin(), v.end());
cout << "After reverse(): ";
for (it = v.begin(); it != v.end(); it++)
{
    cout << *it << " ";
}
cout << endl;

return 0;
}
```

Vectors: Dynamic arrays

- ❑ Vector is a sequence container that stores elements in contiguous memory and can dynamically resize itself.

Differences	Arrays	Vectors
Size	Fixed size (cannot grow or shrink)	Dynamic size (resizes automatically)
Memory	elements usually on stack (unless dynamically allocated)	elements stored on heap
Insertion/Deletion	slow for adding/removing elements in the middle	fast at the end (push_back()), slower elsewhere

Common Functions

- `push_back(value)` → add element at end
- `pop_back()` → remove last element
- `size()` → current number of elements
- `capacity()` → allocated storage size
- `empty()` → check if vector is empty

```
class MyVector {  
private:  
    int* arr;    // pointer to array  
    int sz;      // current size  
    int cap;     // current capacity  
  
    // Resize the internal array  
    void resize(int newCap) {  
        int* newArr = new int[newCap];  
        for (int i = 0; i < sz; i++)  
            newArr[i] = arr[i];  
        delete[] arr;  
        arr = newArr;  
        cap = newCap;  
    }  
}
```

```
public:  
    // Constructor  
    MyVector() {  
        arr = new int[1];  
        sz = 0;  
        cap = 1;  
    }  
  
    // Destructor  
    ~MyVector() {  
        delete[] arr;  
    }  
  
    // Add element at the end  
    void push_back(int value) {  
        if (sz == cap)  
            resize(cap * 2); // double capacity  
        arr[sz++] = value;  
    }  
  
    // Remove last element  
    void pop_back() {  
        if (sz > 0)  
            sz--; }  
}
```

```
// Get current size
```

```
int size() {  
    return sz;  
}
```

```
// Get current capacity
```

```
int capacity() {  
    return cap;  
}
```

```
// Access element by index
```

```
int& operator[](int index) {  
    if (index < 0 || index >= sz) {  
        cout << "Index out of bounds!" << endl;  
        exit(1);  
    }  
    return arr[index];  
}
```

```
// Clear all elements
```

```
void clear() {  
    sz = 0;  
}
```

```
// Get first element
```

```
int& front() {  
    if (sz == 0) {  
        cout << "Vector is empty!" << endl;  
        exit(1);  
    }  
    return arr[0];  
}
```

```
// Get last element
```

```
int& back() {  
    if (sz == 0) {  
        cout << "Vector is empty!" << endl;  
        exit(1);  
    }  
    return arr[sz - 1];  
}  
};
```

List

- ❑ List is a sequence container that stores elements in non-contiguous memory and allows fast insertion and deletion at any position.
- ❖ **Non-contiguous memory:** Elements are stored as nodes linked together
- ❖ **Fast insertion/deletion:** Can add or remove elements anywhere efficiently
- ❖ **Traversal using iterators:** Random access like arrays is not allowed
- ❖ **Dynamic size:** Grows or shrinks automatically as elements are added/removed

Common Functions

- `push_back(value)` → Adds element at the end of the list
- `push_front(value)` → Adds element at the beginning of the list
- `pop_back()` → Removes the last element
- `pop_front()` → Removes the first element
- `size()` → Returns the number of elements in the list

```
class Person {
protected:
    string name;
public:
    void setName(string n) { name = n; }
    void showName() {
        cout << "Name: " << name << endl; }
};

class Student : public Person {
private: list<int> marks;
public:
    void addMark(int m) { marks.push_back(m); }
    void insertMark(int index, int m) {
        if (index < 0 || index > marks.size()) {
            cout << "Index out of range!" << endl;
            return; }
        auto it = marks.begin();
        advance(it, index);
        marks.insert(it, m); }
```

```
void removeMark(int index) {
    if (index < 0 || index >= marks.size()) {
        cout << "Index out of range!" << endl;
        return;
    }

    auto it = marks.begin();
    advance(it, index);
    marks.erase(it);
}

void removeLastMark() {
    if (!marks.empty()) { marks.pop_back(); }
}

void showMarks() {
    cout << "Marks: ";
    for (int m : marks) cout << m << " ";
    cout << endl;
}

int totalMarks() {
    int sum = 0;
    for (int m : marks) sum += m;
    return sum; } };
```

```
int main() {  
    list<Student> students;  
  
    Student s1;  
    s1.setName("Ahmad");  
    s1.addMark(80);  
    s1.addMark(90);  
    s1.insertMark(1, 85);  
  
    Student s2;  
    s2.setName("Ali");  
    s2.addMark(70);  
    s2.addMark(85);  
    s2.addMark(75);  
  
    students.push_back(s1);  
    students.push_back(s2);
```

```
    for (auto it = students.begin(); it != students.end(); it++) {  
        it->showName();  
        it->showMarks();  
        cout << "Total: " << it->totalMarks() << endl;  
        cout << "-----" << endl;  
    }  
  
    auto it = students.begin();  
    it->removeMark(1);  
    cout << "After removing a mark from " << endl;  
    it->showName();  
    it->showMarks();  
    return 0;  
}
```



Thank you